Homework 1 Solutions

Alexander McLain

1. For each situation below, state whether the analysis is **supervised** or **unsupervised** learning. Also classify the data structure as **wide**, **large**, **skinny**, or **messy**. Briefly explain your reasoning.

a. (**5 points**) You have survey data on 500 patients with their responses to 40 lifestyle questions, and you want to group them into lifestyle "clusters.' '

Answer: Since there is no outcome and we are completely focusing on the predictor variables, this is an unsupervised learning problem. I would say this dataset is *skinny*, but not overly so. 40 variables is a lot.

b. (**5 points**) You have 2000 patients with blood pressure, BMI, and smoking status recorded, and you want to predict who will develop heart disease within 5 years.

Answer: There is a clear outcome we are trying to predict based on the predictor variables, thus is a supervised learning problem. This dataset is *skinny*.

c. (**5 points**) You have a genomic dataset with 100 patients and 15,000 gene expression measures, and you want to predict drug response.

Answer: There is a clear outcome we are trying to predict based on the predictor variables, thus is a supervised learning problem. This dataset is *wide* since there are more predictor variables than observations.

d. (**5 points**) You have electronic health records (EHR) for 800 patients from a very busy ER in a large metropolitan area. The data are available for various markers and ICD-9 codes, but are spotty. You want to build a model to predict the length of stay.

Answer: There is a clear outcome we are trying to predict based on the predictor variables, thus is a supervised learning problem. EHR data are a classic example of *messy* data. They are hard to merge, clean, and have lots of missing observations.

2. Below is the code contained in the question:

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
# Load and prepare data
data("birthwt", package = "MASS")
# Remove "low" variable.
bw <- birthwt[,-1]

# Create factors where appropriate
bw$race <- factor(bw$race, labels = c("white","black","other"))
bw$smoke <- factor(bw$smoke, labels = c("nonsmoker","smoker"))
bw$ht    <- factor(bw$ht, labels = c("no","yes"))
bw$ui    <- factor(bw$ui, labels = c("no","yes"))
```

```r
# Define outcome and predictors
outcome <- "bwt"
predictors <- setdiff(names(bw), outcome)

# Generate all subsets up to size 4 (to keep it manageable)
subsets <- unlist(
  lapply(1:4, function(k) combn(predictors, k, simplify = FALSE)),
  recursive = FALSE
)

# Create formulas
formulas <- lapply(subsets, function(vars) {
  as.formula(paste(outcome, "~", paste(vars, collapse = " + ")))
})
```

a. (**10 points**) Fit a standard multiple linear regression with **birth weight** as the outcome. Use adjusted R^2, BIC, and Mallows Cp to explore which predictors are important.

Answer you actually don't need to use the code above to answer this question. As we say "there is more than one way to kill a rat." However, I'll show both versions.

Using the code above

```r
# Fit each model and compute Adjusted R^2, BIC, and Mallows Cp
get_fit_stats <- function(form, data, sigma2_full) {
  fit <- lm(form, data = data)

  # Calculate Mallows Cp
  n <- nobs(fit)                      # sample size
  p <- length(coef(fit))              # number of parameters incl intercept
  sse <- deviance(fit)                # residual sum of squares

  cp <- sse/sigma2_full - (n - 2*p)

  tibble(
    model = deparse(form),
    n_params = p,
    sse = sse,
    adj_r2 = summary(fit)$adj.r.squared,
    bic = BIC(fit),
    cp = cp
  )
}

# Get all fit statistics, then keep only the best model (best sse) for each value of p
# For Mallows get the MSE from the full model
fit_full <- lm(bwt ~ ., data = bw)
sigma2_full <- summary(fit_full)$sigma^2  #

rmse_df <- purrr::map_dfr(formulas, get_fit_stats, data = bw, sigma2_full = sigma2_full)  %>%
  group_by(n_params) %>%
  arrange(sse) %>%
  filter(row_number() == 1)

rmse_df  %>% knitr::kable(digits = 3)
```

model	n_params	sse	adj_r2	bic	cp
bwt ~ race + smoke + ht + ui	6	78611734	0.192	3018.387	8.879
bwt ~ race + smoke + ui	5	81069681	0.171	3018.964	12.691
bwt ~ lwt + ht + ui	4	85191369	0.134	3023.095	20.437
bwt ~ ht + ui	3	88748030	0.103	3025.583	26.847
bwt ~ ui	2	91910625	0.076	3026.960	32.325

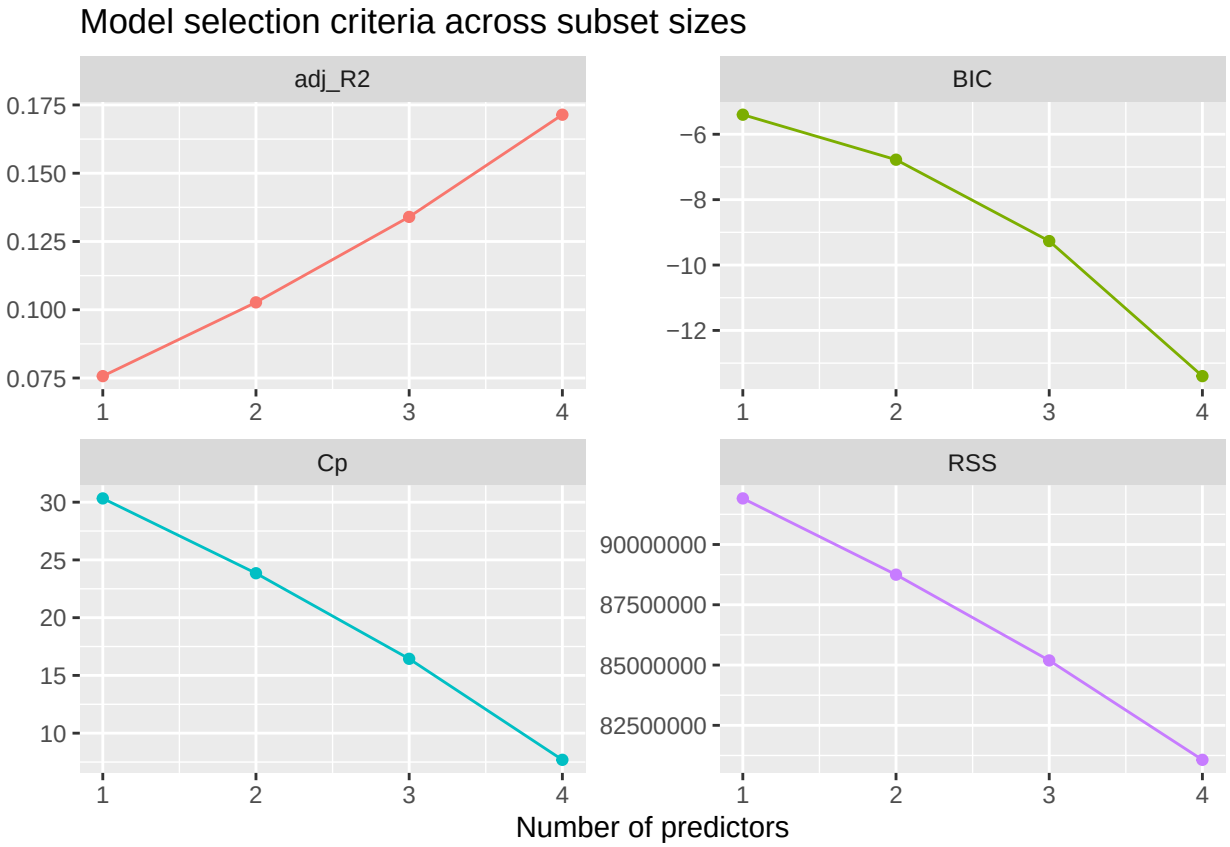
All model fit criteria like the model with all 4 predictors. Here's the coefficients for the best 4 predictor model:

```
best_model <- lm(rmse_df$model[1], data = bw)
coef(best_model, id = 4)  %>% knitr::kable(digits = 3)
```

	x
(Intercept)	3412.764
raceblack	-425.061
raceother	-409.259
smokesmoker	-386.203
htyes	-472.328
uiyes	-563.088

Not using the code above (which is much simplier).

```
library(leaps)
best_subset <- regsubsets(bwt ~ ., bw, nvmax = 4) #only up to 4 since that's what above uses
results <- summary(best_subset)

summ_results <- tibble(
  predictors = 1:4,
  adj_R2 = results$adjr2,
  Cp     = abs(results$cp - 2:5), # Want Cp to be close to p+1
  BIC    = results$bic,
  RSS    = results$rss
)

summ_results %>%
  pivot_longer(-predictors, names_to = "stat", values_to = "value") %>%
  ggplot(aes(predictors, value, color = stat)) +
  geom_line(show.legend = FALSE) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~ stat, scales = "free") +
  labs(x = "Number of predictors", y = NULL,
       title = "Model selection criteria across subset sizes")
```



All model fit criteria like the model with all 4 predictors. Here's the predictors in the best 4 predictor model:

```
coef(best_subset, id = 4)  %>% knitr::kable(digits = 3)
```

	x
(Intercept)	3387.351
raceblack	-455.406
raceother	-417.073
smokesmoker	-393.561
uiyes	-527.659

Note: notice that the answers are not the same! The easier version treats `raceblack` and `raceother` as two separate predictors. Technically, this is correct as the race variable creates two dummy variables. However, I think in practice we would consider `race` to be a single predictor variable (even with dummy coding).

b. (**5 points**) Use backwards selection criteria to find the best model.

```
bf_mod  <- lm(bwt ~ ., data = bw)
back_mf <- stepAIC(bf_mod, direction = "backward", trace = FALSE)

back_mf$coefficients  %>% knitr::kable(digits = 3)
```

	x
(Intercept)	2837.264
lwt	4.242

	x
raceblack	-475.058
raceother	-348.150
smokesmoker	-356.321
htyes	-585.193
uiyes	-525.524

c. (**5 points**) Fit a **forward stagewise regression** model on the same data.

Answer the code for this is similar to what we used in class. However, first we need to create a design matrix from our predictor variables since the `lars` functions don't work with factors.

```
library(lars)
```

```
## Loaded lars 1.3
```

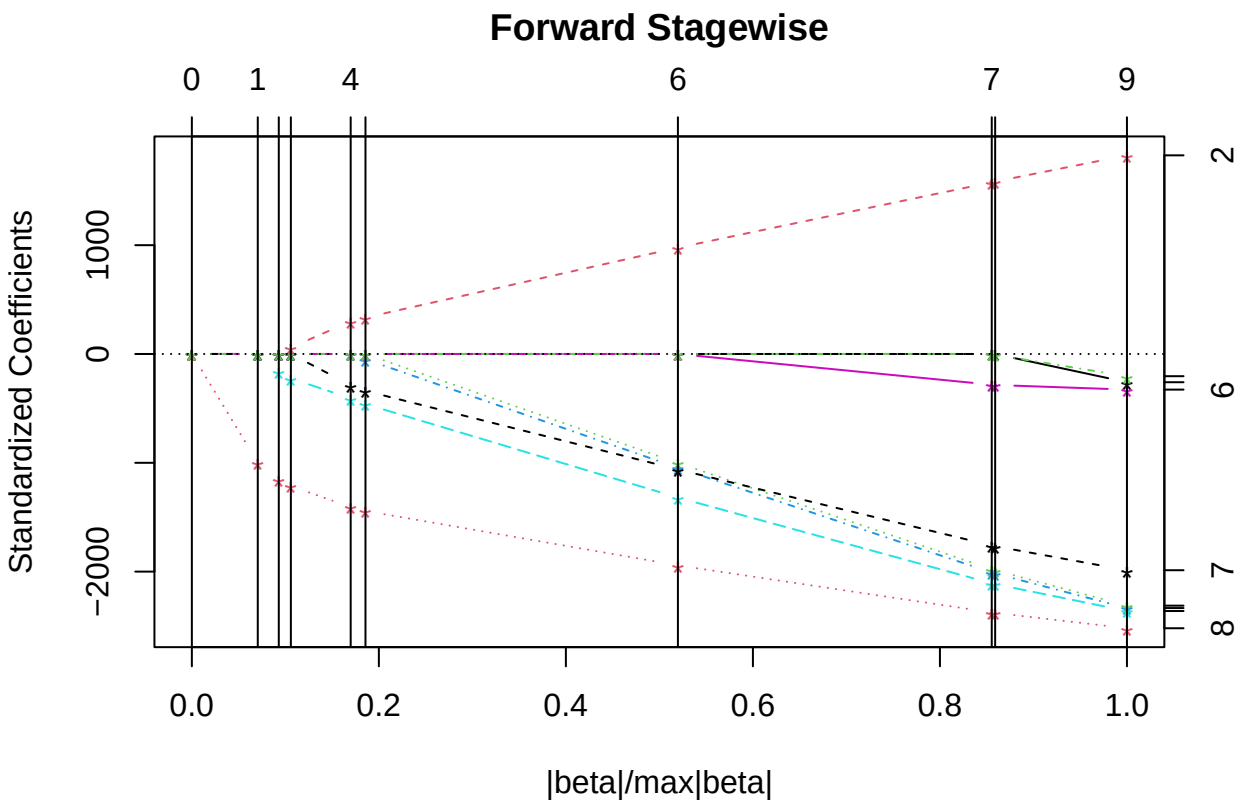
```
# Create design matrix for X
X_only <- bw[,colnames(bw) != "bwt"]
# Get formula for all X variables
X_form <- as.formula(paste("~", paste(colnames(X_only), collapse = " + ")))
X_form
```

```
## ~age + lwt + race + smoke + ptl + ht + ui + ftv
```

```
# Get design matrix for formula (the [,-1] will remove the intercept)
X <- model.matrix(X_form, data = bw)[,-1]
head(X, 3)  %>% knitr::kable(digits = 3)
```

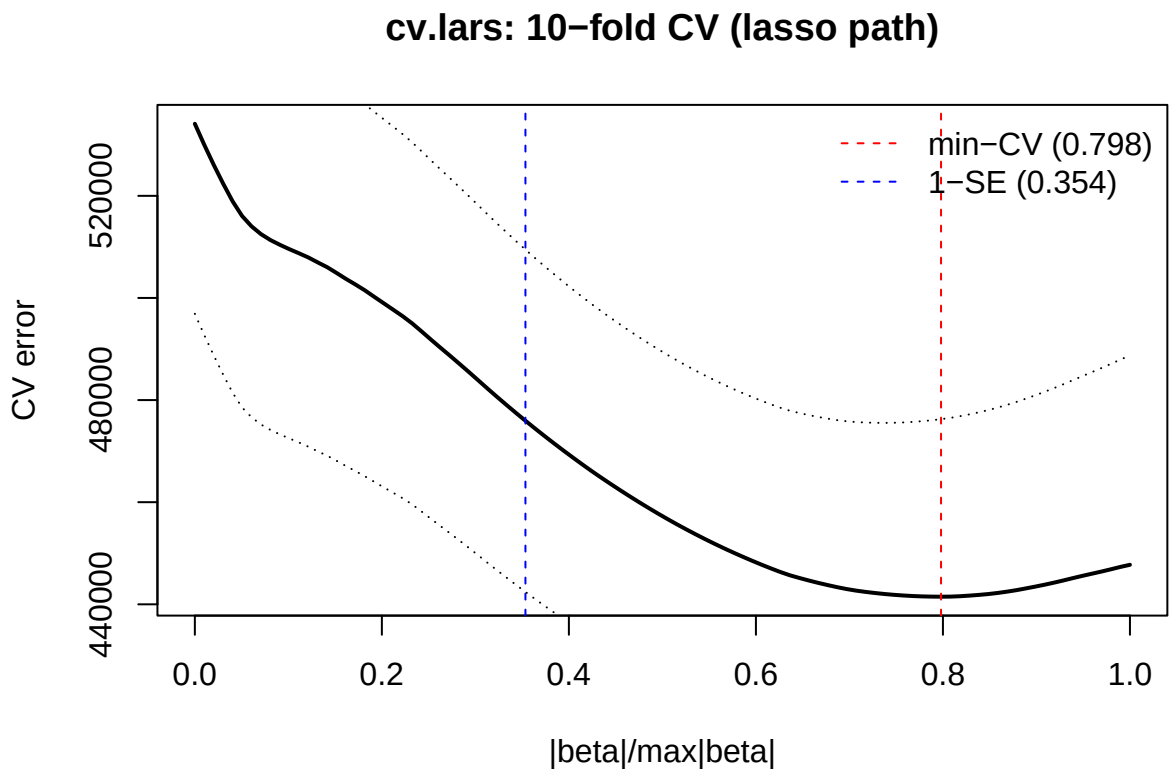
	age	lwt	raceblack	raceother	smokesmoker	ptl	htyes	uiyes	ftv
85	19	182	1	0	0	0	0	1	0
86	33	155	0	1	0	0	0	0	3
87	20	105	0	0	1	0	0	0	1

```
y <- bw$bwt

# 1) Fit the full forward stagewise model
ForStag_mf <- lars(X, y,
                   type = "forward.stagewise", trace = FALSE)
plot(ForStag_mf)
```




```r
# 2) K-fold CV along that path (K = 10 here)
set.seed(735794)
cv_out <- cv.lars(X, y, K = 10, type = "forward.stagewise", plot.it = FALSE, se = TRUE)

# The CV object contains:
# cv_out$cv        -> mean CV error at each fraction
# cv_out$cv.error  -> (approx) SE of CV error at each fraction
# cv_out$index     -> Abscissa values at which the CV curve is be computed.
#                     If mode="fraction" this is the fraction of the saturated |beta|.

# 3) Pick tuning by minimum CV
i_min <- which.min(cv_out$cv)
frac_min <- cv_out$index[i_min]

# 1-SE rule: pick the simplest (largest shrinkage / smaller fraction)
# whose CV error <= cv_min + SE_min.
cv_min <- cv_out$cv[i_min]
se_min <- cv_out$cv.error[i_min]
target <- cv_min + se_min

# Among all fractions with CV <= target, pick the *smallest* index that
# still satisfies this (for lars' fraction scale, smaller fraction = less variables in the model).
i_1se <- min(which(cv_out$cv <= target))
frac_1se <- cv_out$index[i_1se]

# 4) Inspect / plot the CV curve
```

```r
plot(cv_out$index, cv_out$cv, type = "l", lwd = 2,
     xlab = "|beta|/max|beta|", ylab = "CV error",
     main = "cv.lars: 10-fold CV (lasso path)")
lines(cv_out$index, cv_out$cv + cv_out$cv.error, lty = 3)
lines(cv_out$index, cv_out$cv - cv_out$cv.error, lty = 3)
abline(v = c(frac_min, frac_1se), col = c("red","blue"), lty = 2)
legend("topright",
       c(sprintf("min-CV (%.3f)", frac_min),
         sprintf("1-SE (%.3f)", frac_1se)),
       col = c("red","blue"), lty = 2, bty = "n")
```




```r
# 5) Coefficients at chosen fraction (min-CV shown; use frac_1se similarly)
coef_min <- predict(ForStag_mf, s = frac_min, type = "coef", mode = "fraction")$coefficients
coef_min  %>% knitr::kable(digits = 3)
```

	x
age	0.000
lwt	3.518
raceblack	-382.488
raceother	-280.381
smokesmoker	-294.022
ptl	-34.613
htyes	-489.366
uiyes	-471.197
ftv	0.000

```r
coef_1se <- predict(ForStag_mf, s = frac_1se, type = "coef", mode = "fraction")$coefficients
coef_1se  %>% knitr::kable(digits = 3)
```

	x
age	0.000
lwt	1.568
raceblack	-106.198
raceother	-83.532
smokesmoker	-132.919
ptl	0.000
htyes	-209.385
uiyes	-346.792
ftv	0.000

The 1se model is similar to the previous models we've found in terms of varibles in the model. It has the same variables as the backwards model, but different coefficient estimates. The "min" model includes more variables than any of the models we've used before.

d. (**10 points**) Perform 10-fold **cross-validation** to evaluate predictive error. Instead of mean squared error, use the **Median Absolute Deviation (MAD)**:

$$MAD = \text{median}\left(|y_i - \hat{y}_i|\right).$$

MAD is sometimes preferred to MSE when there are outliers in the data. MAD is less sensitive to outliers than MSE is.

Answer: To implement this we are going to alter the code from class to use MAD instead of RMSE to select the best model.

Previously, we used the `yardstick` package to estimate the rmse via the `rmse_vec` function. `yardstick` has a mean absolute error (`mae`) function, but not an MAD function. So, we'll create one.

```r
library(rsample)    # splitting, vfolds

#Create function to calculate mad
mad_vec <- function(obs,pred){
  median(abs(obs - pred))
}

# Function to implement the CV
cv_fit <- function(form_str, folds){
  form <- as.formula(form_str)
  # compute MAD across folds
  mad_vals <- map_dbl(folds$splits, function(s){
    tr <- analysis(s); va <- assessment(s)
    fit <- lm(form, data = tr)
    pred <- predict(fit, newdata = va)
    mad_vec(va$bwt, pred)
  })
  tibble(
    model = Reduce(paste, deparse(form)),
    cv_mad = mean(mad_vals),
    cv_sd = sd(mad_vals)
  )
```

```
}

# Implement the CV
set.seed(2024)
folds <- vfold_cv(bw, v = 10)

cv_res <- formulas %>%  purrr::map_dfr(cv_fit, folds = folds) %>%
  arrange(cv_mad)

cv_res %>% slice_head(n = 10) %>% knitr::kable(digits = 3, caption = "Top 10 candidates by 10-fold CV (
```

Table 8: Top 10 candidates by 10-fold CV (training set only)

model	cv_mad	cv_sd
bwt ~ race + smoke + ht + ui	455.528	124.354
bwt ~ lwt + race + smoke + ui	465.983	108.302
bwt ~ age + race + smoke + ui	470.041	108.493
bwt ~ race + smoke + ui	474.467	112.573
bwt ~ age + lwt + smoke + ui	475.971	92.791
bwt ~ race + smoke + ht + ftv	477.691	132.448
bwt ~ age + smoke + ht + ui	480.736	155.073
bwt ~ lwt + smoke + ui	482.588	90.862
bwt ~ race + smoke + ui + ftv	482.692	112.480
bwt ~ race + smoke + ht	482.776	129.920

e. (**15 points**) Perform either:
- **Nested CV**, or
- **CV with a separate test set**

to both select the best model *and* to estimate its prediction error (still using MAD).

Answer: I'll show both options here.

For the nested CV, we can alter the code used in class similarly to how we did this in d.

```
set.seed(2025)
outer_folds <- vfold_cv(bw, v = 10)

nested_scores <- map_dbl(outer_folds$splits, function(outer_split){

  tr_outer <- analysis(outer_split)
  te_outer <- assessment(outer_split)

  # inner CV on the outer training split
  inner_folds <- vfold_cv(tr_outer, v = 10)

  # run inner CV across the same grid
  inner_cv_res <- formulas %>%  purrr::map_dfr(cv_fit, folds = inner_folds) %>%
    arrange(cv_mad)

  # choose best by inner CV
  best_form <- as.formula(inner_cv_res$model[1])

  # refit on tr_outer, evaluate on te_outer (outer hold-out)
  fit_outer <- lm(best_form, data = tr_outer)
```

```
  mad_vec(te_outer$bwt, predict(fit_outer, newdata = te_outer))
})

nested_cv_mad <- median(nested_scores)
```

Now we'll use the other approach. This will use the code from d, but only evaluated on a training set.

```
# Train/test split
set.seed(42)
split <- initial_split(bw, prop = 2/3, strata = NULL)
train <- training(split)
test  <- testing(split)

# Implement the CV on the train
set.seed(2024)
folds <- vfold_cv(train, v = 10)

cv_res_tr <- formulas %>%  purrr::map_dfr(cv_fit, folds = folds) %>%
  arrange(cv_mad) %>% slice_head(n = 1)

# Here's the best model:
cv_res_tr$model[1]
```

```
## [1] "bwt ~ race + smoke + ht + ui"
```

```
# Refit the best model to the whole training set.
best_form <- as.formula(cv_res_tr$model[1])
fit_best_tr <- lm(best_form, data = train)

# Predict for the test set and calculate the MAD
pred_test <- predict(fit_best_tr, newdata = test)
test_MAD <- mad_vec(test$bwt, pred_test)
```

Now, we'll compare the estimated MAD from nested CV, a test set after CV, and the best MAD from CV.

```
tibble(
  Estimate = c("Nested CV MAD", "MAD on test set after CV", "Winner's CV MAD (from d.)"),
  Value    = c(nested_cv_mad, test_MAD, cv_res$cv_mad[1])
) %>% knitr::kable(digits = 3)
```

Estimate	Value
Nested CV MAD	480.675
MAD on test set after CV	459.981
Winner's CV MAD (from d.)	455.528

We'd expect the winner's CV to be the most biased.

f. (**5 points**) Compare results across all approaches and write a short paragraph on which model you would choose and why.

Various answers accepted.

2. We will use the `airquality` dataset (built into R), which records daily air quality measurements in New York, May–September 1973.

a. (**5 points**) Fit a linear regression model predicting **Ozone** levels using predictors you think are appropriate (e.g., Solar, Wind, Temp, Month). You don't need to perform any model selection; simply select a model that you think is appropriate and use it for the remainder of this question.

Answer: Obviously there are a lot of different options here, but I'll use up to a third degree polynomial for all variables.

```
library(tidyverse)
data(airquality)
## Remove missing values
airquality_cc <- airquality[complete.cases(airquality),]

formula <- Ozone ~ poly(Solar.R,3) + poly(Wind,3) + poly(Temp,3) + poly(Month,3) + poly(Day,3)

fit_airqual <- lm(formula, data = airquality_cc)
```

b.(**15 points**) Check the key regression assumptions:

- Linearity (residual plots, component+residual plots),
- Homoscedasticity (residuals vs fitted, Breusch–Pagan test),
- Normality of residuals (QQ plot),
- Influence (Cook's distance, leverage).

Here, I'm going to follow things close to the example in class.

Augmented residual data

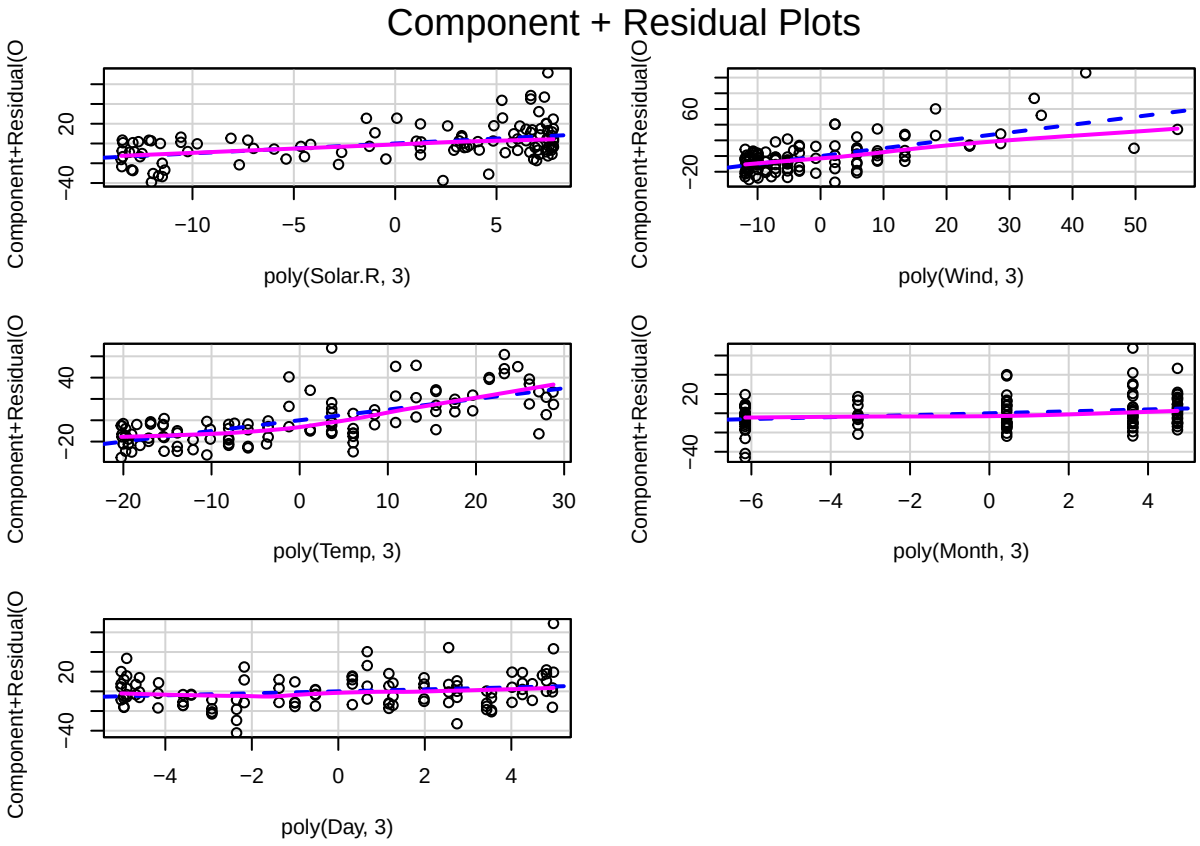
```
library(broom)     # augment for residuals, leverage, cooks distance
aug <- augment(fit_airqual)
head(aug,3)
```

```
## # A tibble: 3 x 13
##   .rownames Ozone `poly(Solar.R, 3)`[,"1"]  [,"2"]  [,"3"] `poly(Wind, 3)`[,"1"]
##   <chr>     <int>                    <dbl>   <dbl>   <dbl>                 <dbl>
## 1 1            41                  0.00544 -0.101  -0.0138               -0.0681
## 2 2            36                 -0.0699  -0.0848  0.131                -0.0520
## 3 3            12                 -0.0374  -0.108   0.0781                0.0713
## # i 10 more variables: `poly(Wind, 3)`[2:3] <dbl>, `poly(Temp, 3)` <poly[,3]>,
## #   `poly(Month, 3)` <poly[,3]>, `poly(Day, 3)` <poly[,3]>, .fitted <dbl>,
## #   .resid <dbl>, .hat <dbl>, .sigma <dbl>, .cooksd <dbl>, .std.resid <dbl>
```

1) Linearity: Component + Residual (Partial Residual) Plots

If relationships are strongly curved, consider transformations or adding terms (splines, polynomials).

```r
car::crPlots(fit_airqual)  # opens a panel of C+R plots
```

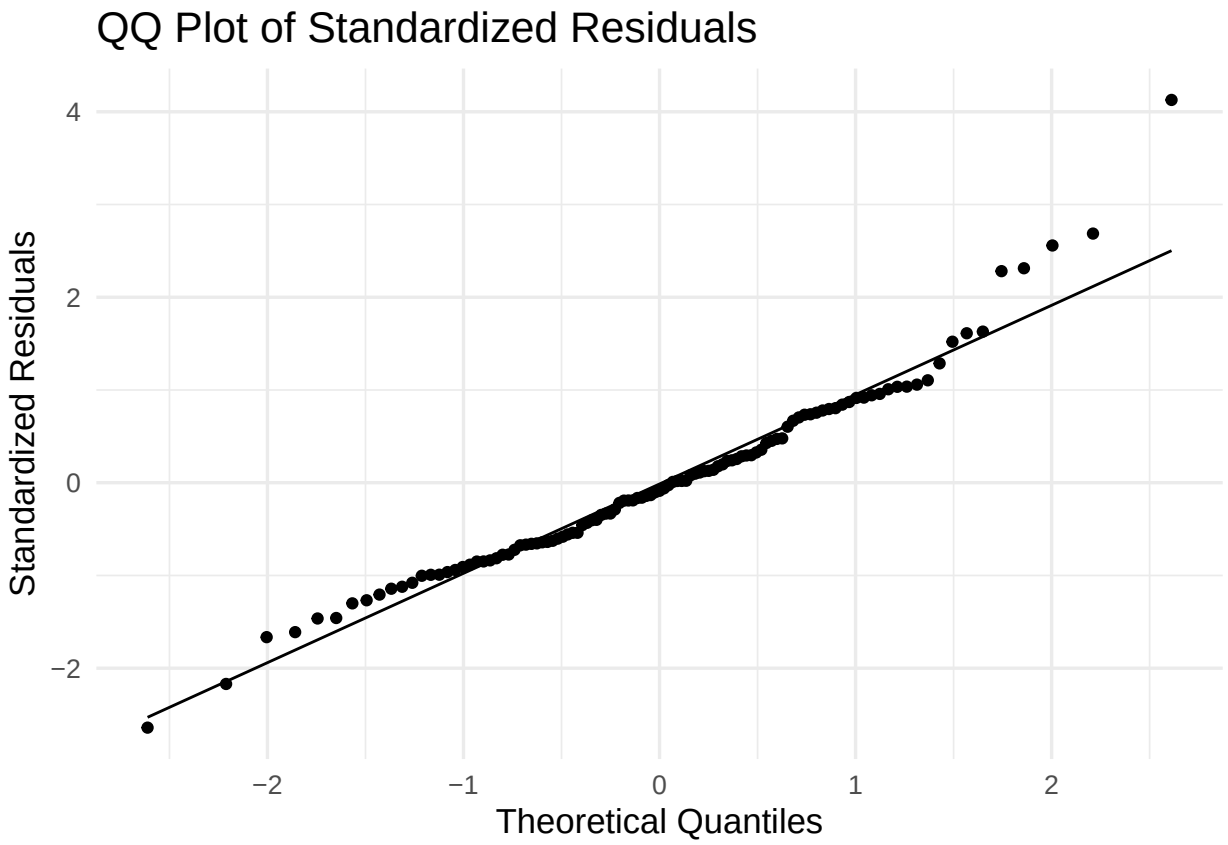


I've already been pretty flexible in terms of how these variables can relate to the outcome, so it's no surprise that all of these show relatively linear fits.

2) Normality of Residuals (Primarily for CIs/p-values)

For prediction, normality is less critical than calibration and unbiasedness, but we inspect QQ plot.

```r
ggplot(aug, aes(sample = .std.resid)) +
  stat_qq() + stat_qq_line() +
  labs(title = "QQ Plot of Standardized Residuals",
       x = "Theoretical Quantiles", y = "Standardized Residuals") +
  theme_minimal(base_size = 13)
```



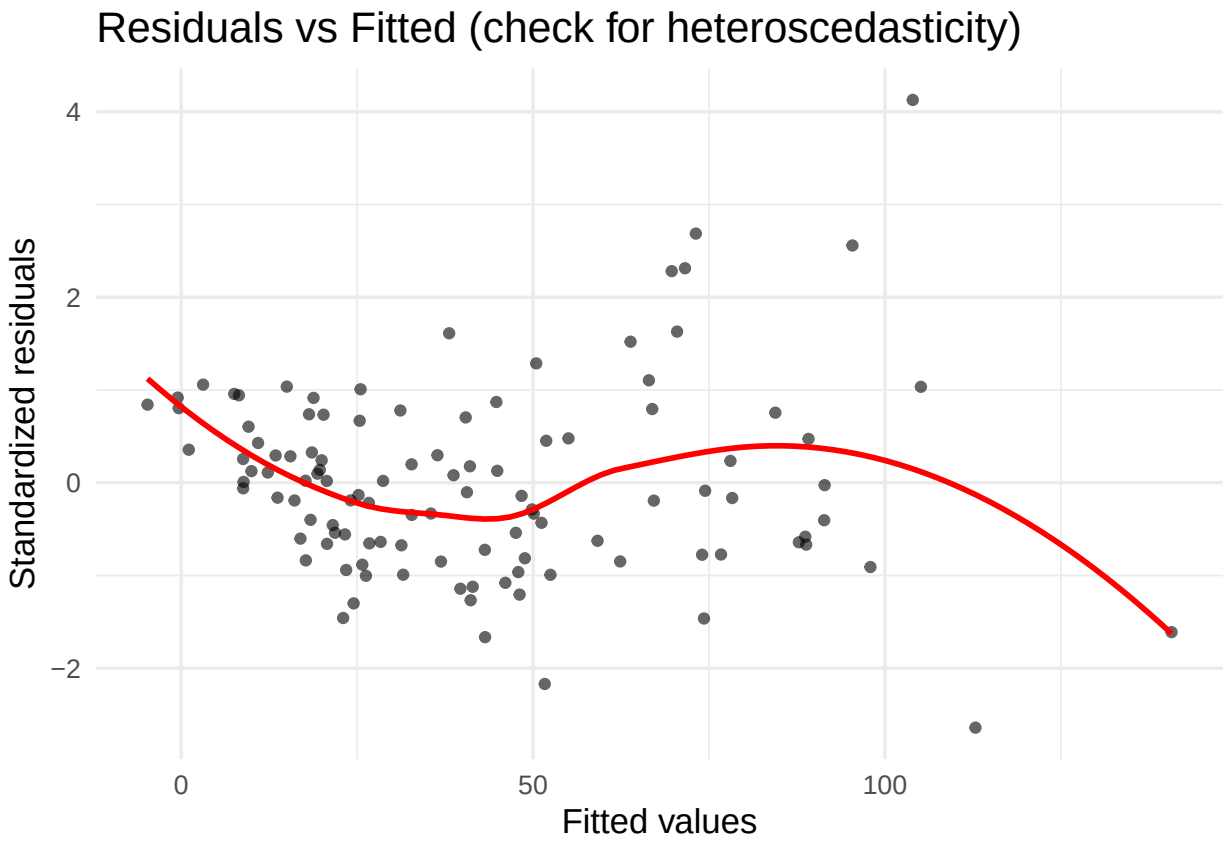
There appears to be one outlier here (on the right side). Beside that, the normal assumption looks great.

3) Homoscedasticity (Constant Error Variance)

Look for funnel shapes; the red LOESS line should be roughly flat.

```
ggplot(aug, aes(.fitted, .std.resid)) +
  geom_point(alpha = 0.6) +
  geom_smooth(se = FALSE, color = "red") +
  labs(x = "Fitted values", y = "Standardized residuals",
       title = "Residuals vs Fitted (check for heteroscedasticity)") +
  theme_minimal(base_size = 13)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Based on this plot, there is clear signs that the variance of the residuals is increasing with the mean. This is common in data that are bounded below by zero.

Let's confirm with the Breusch-Pagan test:

```
lmtest::bptest(fit_airqual)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  fit_airqual
## BP = 37.634, df = 15, p-value = 0.001022
```

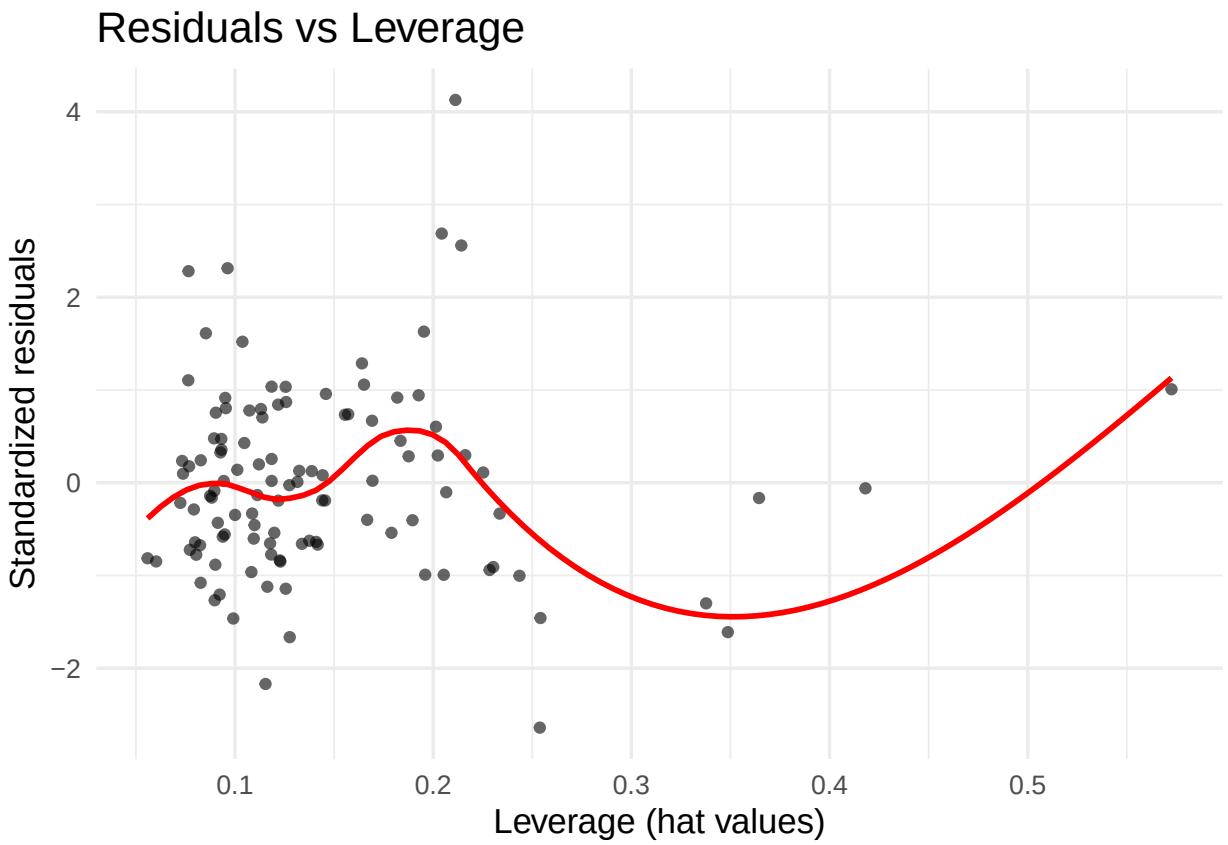
I would (generally) refit the model using $\log(Ozone)$ instead of ozone.

4) Influence: Leverage & Cook's Distance

Identify points that overly influence the fit.

```
ggplot(aug, aes(.hat, .std.resid)) +
  geom_point(alpha = 0.6) +
  geom_smooth(se = FALSE, color = "red") +
  labs(x = "Leverage (hat values)", y = "Standardized residuals",
       title = "Residuals vs Leverage") +
  theme_minimal(base_size = 13)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
# Top potentially influential points by Cook's D
aug %>%
  mutate(cook = .cooksd) %>%
  arrange(desc(cook)) %>%
  slice_head(n = 12) %>%
  dplyr::select(.fitted, .std.resid, .hat, cook) %>%
  knitr::kable(digits = 3, caption = "Top 10 observations by Cook's distance")
```

Table 10: Top 10 observations by Cook's distance

.fitted	.std.resid	.hat	cook
103.949	4.128	0.211	0.285
112.859	-2.641	0.254	0.148
73.127	2.687	0.204	0.116
95.376	2.558	0.214	0.112
140.719	-1.611	0.349	0.087
25.487	1.008	0.573	0.085
24.497	-1.301	0.338	0.054
23.011	-1.459	0.254	0.045
70.461	1.629	0.195	0.040
51.657	-2.170	0.115	0.038
71.587	2.313	0.096	0.036
69.696	2.281	0.077	0.027

Note: the common cutoff for Cook's D is $4/n$. I'm not sure where this cutoff came from, but in my experience

this cutoff results in around 10% of the sample being labeled as an "outlier". Here, $4/n = 0.036$ would result in 11 outliers out of 111.

What's more important than the cufoff is to look at values that are far away from others. Let's look at the data for the person with the largest Cook's distance.

```
aug%>%
  mutate(cook = .cooksd) %>%
  arrange(desc(cook)) %>%
  slice_head(n = 1) %>%
  dplyr::select(Ozone,.fitted:cook)
```

```
## # A tibble: 1 x 8
##   Ozone .fitted .resid  .hat .sigma .cooksd .std.resid  cook
##   <int>   <dbl>  <dbl> <dbl>  <dbl>   <dbl>      <dbl> <dbl>
## 1   168    104.   64.1 0.211   15.9   0.285       4.13 0.285
```

```
airquality %>%
  filter(Ozone == 168)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1   168     238  3.4   81     8  25
```

This day has the highest ozone measurement. As I said above, I would transform the outcome (in real life), where this observation wouldn't be an issue.

c. (**10 points**) Estimate the **prediction error (MAD)** of your model separately for each **Month**. Report MAD in May, June, July, August, and September. Comment on whether the model performs equally well across months, and what that might suggest about seasonal variation.

Answer: When we have data that have a temporal structure we should respect that structure when doing cross-validation. That is, the error for August, for example, should not be obtained from a model that contains September. As a result, to get the error for August, we should only use the data from May through July.

However, this is challenging to uphold in this situation because someone (*me*) asked for the error for May, and May is the first month in the training set. We can give the error for May without using data from beyond May.

As a result, I'm going to do this the *right* way but only for June through September. For this model, I'm only going to use a linear effect of `Month` since the 3rd degree polynomial will only work when I have 3 different months. When only May is in the training set, I can't have any effect of Month.

```
MAD_error_seq <- function(mon){

  # Setting the model formula separately for June versus the others.
  if(mon == 6){
    formula <- Ozone ~ poly(Solar.R,3) + poly(Wind,3) + poly(Temp,3) + poly(Day,3)
  }else{
    formula <- Ozone ~ poly(Solar.R,3) + poly(Wind,3) + poly(Temp,3) + Month + poly(Day,3)
  }
  tr <- airquality_cc %>%
    filter(Month < mon)
  tst <- airquality_cc %>%
    filter(Month == mon)
  fit <- lm(formula, data = tr)
  pred <- predict(fit, newdata = tst)
  tibble(
    Month = mon,
```

```
    mad = mad_vec(tst$Ozone, pred)
  )
}

seq_results <- map_dfr(6:9, MAD_error_seq) %>%
  mutate(Error_type = "Sequential")

seq_results %>% knitr::kable(digits = 2, caption = "Error using the sequential nature.")
```

Table 11: Error using the sequential nature.

Month	mad	Error_type
6	17.81	Sequential
7	16.05	Sequential
8	14.61	Sequential
9	12.17	Sequential

Notice that the mad is decreasing as we get more months of data. This is probably because the model has more data to train from. The estimate for September, is the best estimate of the *"mad when all data except one month is used"*.

In the following version, I will do the incorrect thing and ignore the sequential nature of the data. I'm still going to use the same model formulas as above, so we can compare them.

```
MAD_error_not_seq <- function(mon){

  # Setting the model formula separately for June versus the others.
  if(mon == 6){
    formula <- Ozone ~ poly(Solar.R,3) + poly(Wind,3) + poly(Temp,3) + poly(Day,3)
  }else{
    formula <- Ozone ~ poly(Solar.R,3) + poly(Wind,3) + poly(Temp,3) + Month + poly(Day,3)
  }
  tr <- airquality_cc %>%
    filter(Month != mon)
  tst <- airquality_cc %>%
    filter(Month == mon)
  fit <- lm(formula, data = tr)
  pred <- predict(fit, newdata = tst)
  tibble(
    Month = mon,
    mad = mad_vec(tst$Ozone, pred)
  )
}

not_seq_results <- map_dfr(5:9, MAD_error_not_seq) %>%
  mutate(Error_type = "Not Sequential")

seq_results %>%
  bind_rows(not_seq_results) %>%
  arrange(Month) %>%
  knitr::kable(digits = 2, caption = "Error using the sequential nature.")
```

Table 12: Error using the sequential nature.

Month	mad	Error_type
5	14.05	Not Sequential
6	17.81	Sequential
6	11.86	Not Sequential
7	16.05	Sequential
7	12.59	Not Sequential
8	14.61	Sequential
8	12.50	Not Sequential
9	12.17	Sequential
9	12.17	Not Sequential

Clearly there's a big difference here. The differences are due to two things:

(1) the non-sequential uses more data (except for September), and
(2) the non-sequential can use data from before and after the month.

As much as I would like to stress the importance of (2), I think (1) plays a bigger role in the differences. As I said above, the mad for September (12.17) is the best estimate of the *"mad when all data except one month is used"*. Only one of the values for the non-sequential approach are below this level (for June). Given that and how much the mad decreases in the sequential approach, I think (1) is playing a bigger role.

That being said, when doing any type of validation clustering and time sequencing are important facets to pay attention to.